BrainAI

ProSuite Industrial

PROSUITE INDUSTRIAL

PRODUCT DOCUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 PUBLIC CLASSIFICATION
Core Architecture & Development	**Technical Documentation**	Access Level: Public / Beta

---

Revision History

Version	Date	Author	Status	Description
1.0.0	2026-03-01	S. Köhne	Stable	Initial Release: ProTU Test Coverage Specification

1. Introduction & Audit Objectives

The **Process Trust Unit (ProTU)** is the automated validation framework for the ProSuite Industrial. This document specifies the exact test vectors, mathematical tolerances, and physical integrity checks that ProTU performs on the target system during an audit run (`protu_industrial ALL`).

The objective of this document is to provide IT security officers and external auditors (e.g., for ISO 27001 or TISAX) with transparent insight into the metrics of the certification process.

2. Module Specifications (User-Space)

The following cryptographic and logical tests are performed for the core modules of the ProSuite.

2.1 ProKey (Hardware Entropy Engine)

ProKey is responsible for generating physical randomness (entropy). A failure here compromises all downstream keys.

- **Sanity & Starvation Check**: Verifies if the random number generator is "dead" (returning only zero-bytes).
- **Repetition Check (Stuck-At Faults)**: Generates sequential keys (K_1, K_2, K_3) and ensures that no key equals the previous one ($K_n \neq K_{n+1}$). This detects hardware loops or buffer freezes.
- **Statistical Monobit Test**: Generates a 4-kilobyte sample (32,768 bits) and measures the exact distribution of ones and zeros. The test enforces a strict industrial-grade uniform distribution with a tolerance range of 48% to 52%. If the value falls outside this range, a "biased output" is detected, and the audit fails.

2.2 ProHash (Deterministic Integrity Engine)

ProHash provides the incorruptible cryptographic fingerprint for files and data streams.

- **Determinism Verification**: Ensures that identical inputs strictly produce the same 256-bit hash, unaffected by memory states or external factors.
- **Avalanche Effect (Diffusion Analysis)**: A single bit in a 64-byte data block is flipped. ProTU verifies that approximately 50% (tolerance: 35% to 65%) of the resulting 256 hash bits change accordingly. This proves the irreversibility of the algorithm.
- **Streaming Consistency (Sponge Test)**: Validates internal buffer integrity. The test ensures that hashing a complete string yields the same result as hashing the same string passed in multiple asynchronous chunks.
- **Collision Resistance**: Basic check against trivial hash collisions with minimally differing input strings.

2.3 ProED (Encryption Core)

The core algorithm for symmetric high-speed encryption.

- **Basic Roundtrip**: Encrypts a dataset in memory and decrypts it again. The result must match the original bit-for-bit, while the intermediate ciphertext must differ significantly.
- **IV Impact Analysis**: Two identical datasets are encrypted using the same master key but with initialization vectors (IV) differing by only 1 bit. ProTU ensures that two completely uncorrelated cipher streams are created, preventing "Two-Time-Pad" vulnerabilities.
- **Context Ratcheting & Isolation**: Protects against ECB (Electronic Codebook) patterns. Two identical plaintext blocks fed into the same stream sequentially must result in different ciphertexts.

2.4 ProChat (Zero-Trust Protocol)

The application layer for decentralized P2P communication.

- **Tamper Protection (AEAD / HMAC)**: An encrypted packet is intentionally manipulated at the bit level (simulating a Man-in-the-Middle attack). ProTU verifies that the receiver strictly rejects this packet (Auth-Error) instead of processing corrupted data.
- **Replay Attack Protection**: A valid, successfully decrypted packet is captured and resent to the receiver. The test only passes if the system detects the duplication and blocks the packet.

3. Black-Box & System Integration Tests

In addition to the C libraries, ProTU audits the compiled command-line tools (CLI) and Web bridges (WASM) as used by the end user.

- **CLI Parameter Handling**: Tools like `proedc.exe` are intentionally called with missing keys, non-existent files, or incorrect length specifications. ProTU verifies that the tools return secure exit codes (e.g., `1` or `2`) and do not trigger critical system errors.
- **WASM Boundary Defense**: Simulates null-pointer injections and buffer overflows in the WebAssembly heap. This ensures that manipulated JavaScript calls from the browser cannot crash the cryptographic engine (Segfault protection).

4. ProTU Meta-Validation (Self-Test)

An audit tool must be able to prove its own integrity. At the beginning of every audit, ProTU performs a self-test:

- **Counter Mechanics**: Verifies that internal metrics (passed/failed counters) work with mathematical consistency.
- **Logging Overflow Protection**: A massive string is injected into the logging engine to prove that the report generator is immune to buffer overflows.
- **Dual-Seal Linkage**: Checks the cryptographic connection to the ProHash engine required for the final sealing (tamper protection) of the `.brep` audit report.

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**
System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**